Divisão e Conquista

Integrantes: Pedro Lucas de Oliveira Duarte e Samuel Batista Oliveira
Data: 28 de outubro de 2026

Resumo

Comparamos força bruta e divisão e conquista no subarranjo de soma máxima (Etapa 1) e no par de pontos mais próximos (Etapa 2), com testes de corretude e medições em Java. Nos dois problemas a força bruta vence em entradas pequenas e a divisão e conquista passa à frente a partir de n = 400, chegando a cerca de 255 vezes na Etapa 1 (n = 51.200) e 50,5 vezes na Etapa 2 (n = 6.400). O algoritmo de Kadane, de custo linear, foi o mais rápido em todos os tamanhos, e o limiar 16 teve o melhor desempenho global na Etapa 2.

1. Introdução e objetivos

Este trabalho compara implementações por força bruta e divisão e conquista em dois problemas: subarranjo de soma máxima e par de pontos mais próximos. O objetivo é verificar a correção das soluções e comparar, por medições experimentais, o crescimento do tempo de execução com a complexidade assintótica prevista.

2. Metodologia

As implementações foram desenvolvidas em Java. Os tempos foram medidos para entradas crescentes, com cinco repetições por configuração, e os resultados são apresentados como médias. Os dados brutos estão no repositório em arquivos CSV, e os gráficos foram produzidos a partir dessas medições.

Etapa 1. As entradas foram vetores pseudoaleatórios de inteiros entre −1000 e 1000, com semente fixa, aquecimento de 30 execuções e medição por `System.nanoTime`. Ambiente: AMD Ryzen 5 5600GT, 16 GB de RAM, Windows 10 e Java 21.

Etapa 2. Os CSVs registram cinco repetições por tamanho de entrada e por limiar. Ambiente: AMD Ryzen 5 5600GT com Radeon Graphics (6 núcleos, 12 threads, 3,6 GHz), 16 GB de RAM, Windows 11 Pro (compilação 26200) e Java 21.

3. Visão geral dos algoritmos

A Tabela 1 resume os métodos implementados e a complexidade de tempo de cada um. As seções seguintes detalham as recorrências, os testes e as evidências experimentais de cada etapa.

Tabela 1. Algoritmos implementados e complexidade de tempo.

Problema	Método	Complexidade
Subarranjo máximo	Força bruta	$\Theta(n^2)$
Subarranjo máximo	Divisão e conquista	$\Theta(n \log n)$
Subarranjo máximo	Kadane (desafio)	$\Theta(n)$
Pontos mais próximos	Força bruta	$\Theta(n^2)$
Pontos mais próximos	Divisão e conquista	$\Theta(n \log n)$

4. Etapa 1 — Subarranjo de soma máxima

4.1 Algoritmos e recorrências

Força bruta. Examina os n(n+1)/2 intervalos possíveis, acumulando a soma incrementalmente. Cada atualização tem custo constante, e o total é $\Sigma(n-i) = n(n+1)/2 = \Theta(n^2)$ de tempo, com $\Theta(1)$ de espaço auxiliar.

Divisão e conquista. Divide o vetor em duas metades, resolve os melhores intervalos de cada lado e calcula, em tempo linear, o melhor intervalo que cruza o meio. A recorrência é

$$T(n) = 2T(n/2) + \Theta(n), \text{ com } T(1) = \Theta(1).$$

Resolução. Pelo Teorema Mestre, a = 2, b = 2 e $f(n) = \Theta(n) = \Theta(n^{(\log_2 2)})$, o que corresponde ao caso 2, logo $T(n) = \Theta(n \log n)$. Por expansão, $T(n) = cn + 2T(n/2) = 2cn + 4T(n/4) = \ldots = cn \cdot \log_2 n + n \cdot T(1)$: são $\log_2 n$ níveis de recursão, cada um com custo total cn.

Kadane. Percorre o vetor uma única vez, decidindo se estende o intervalo corrente ou reinicia na posição atual. Cada posição custa uma comparação e uma soma, e a relação $T(n) = T(n-1) + \Theta(1)$ se expande em $T(n) = c \cdot n + T(0) = \Theta(n)$, com espaço $\Theta(1)$.

4.2 Testes e critérios de desempate

A Etapa 1 documenta 14 casos de teste, todos aprovados nas três implementações. Os casos de borda e o motivo de cada um são:

- **Vetores somente negativos:** a resposta é o maior elemento isolado, e não um intervalo; falha se o algoritmo assumir que a soma vazia (zero) é válida.
- **Tamanho 1:** é o caso base da recursão; garante que a divisão não tente dividir um vetor indivisível.
- **Zeros e empates:** vários intervalos têm a mesma soma máxima, e o resultado deve seguir a regra de desempate (menor índice inicial e, persistindo o empate, menor índice final), mantendo as três implementações consistentes.
- **Intervalo que cruza o meio:** só é encontrado pela etapa de combinação da divisão e conquista, a parte mais sujeita a erros de índice.
- **Resposta contida em uma das metades:** garante que a combinação não sobrepõe uma resposta correta já encontrada pela recursão.

4.3 Resultados experimentais

A Tabela 2 reúne as médias de tempo para alguns tamanhos representativos. A última coluna indica quantas vezes a divisão e conquista foi mais rápida que a força bruta (valores abaixo de 1 indicam o contrário).

Tabela 2. Tempos médios da Etapa 1 (ms).

n	Força bruta	Div. e conquista	Kadane	Razão FB ÷ D&C
100	0,0024	0,0052	0,0004	0,46
400	0,0345	0,0211	0,0008	1,64
1.600	0,3918	0,0693	0,0016	5,65
6.400	4,6476	0,2149	0,0044	21,6
51.200	285,1313	1,1186	0,0221	254,9

Tabela resumida das médias; a tabela integral e os desvios padrão estão em `figuras/tabela_etapa1.md`.

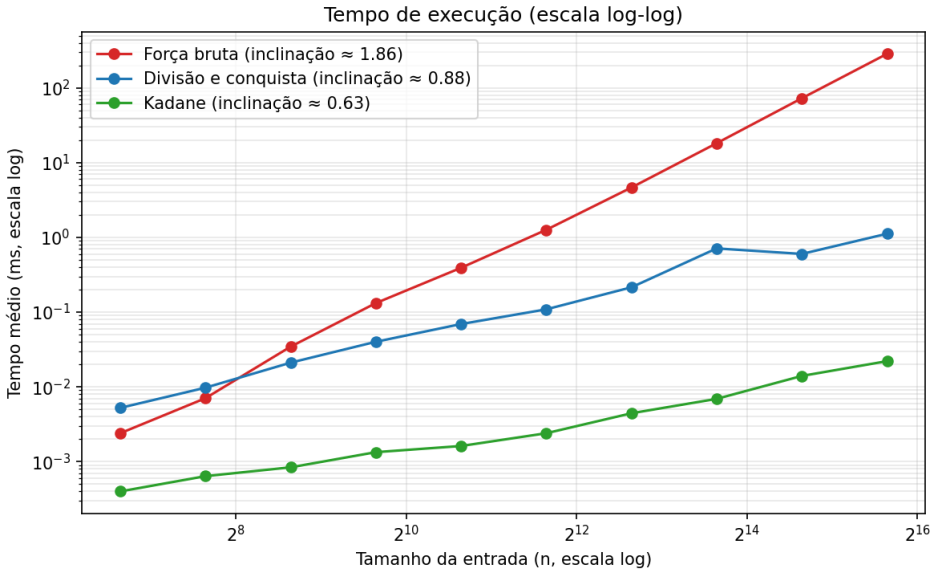


Figura 1. Comparação de tempos em escala log-log (Etapa 1).

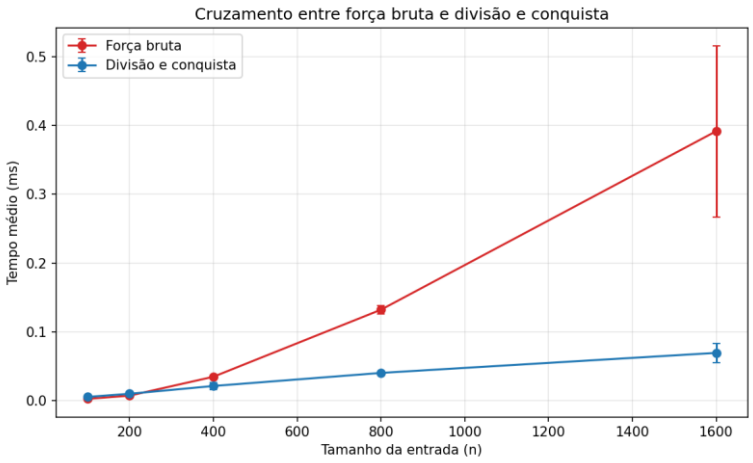


Figura 2. Região de cruzamento entre força bruta e divisão e conquista.

Nos menores tamanhos (n = 100 e 200), a força bruta foi mais rápida por ter menor custo fixo. A partir de n = 400, a divisão e conquista passou a vencer. Em n = 51.200, os tempos médios foram 285,1313 ms contra 1,1186 ms, aproximadamente 255 vezes de vantagem. Kadane foi o mais rápido em todos os tamanhos testados.

Comparação com a teoria. Na força bruta, multiplicar n por 8 (de 6.400 para 51.200) deveria multiplicar o tempo por 64 se o custo fosse $\Theta(n^2)$; o observado foi 61,4 (4,6476 → 285,1313 ms), e a inclinação de 1,86

na Figura 1 é próxima de 2. Já a divisão e conquista (inclinação 0,88) e Kadane (0,63) cresceram menos do que o previsto ($\Theta(n \log n) \approx 1$ e $\Theta(n) = 1$). Esses métodos gastam tempos muito curtos, em que custos fixos, aquecimento da JVM, cache, resolução do relógio e pausas de coleta de lixo pesam mais, achatando a curva nos tamanhos pequenos. Assim, oscilações pontuais não invalidam a análise assintótica.

5. Etapa 2 — Par de pontos mais próximos

5.1 Definição e algoritmos

Dadas n coordenadas bidimensionais, procura-se a menor distância euclidiana entre dois pontos. A força bruta compara todos os pares: $n(n-1)/2$ distâncias, portanto $\Theta(n^2)$ de tempo.

A divisão e conquista ordena os pontos, divide-os pela mediana de x, resolve recursivamente cada metade e considera a menor distância δ encontrada. Em seguida examina a faixa de largura 2δ centrada na divisão, com pontos ordenados por y. A recorrência da etapa recursiva é

$$T(n) = 2T(n/2) + \Theta(n),$$

cuja solução é a mesma da Etapa 1 (Teorema Mestre, caso 2): $\Theta(n \log n)$. Isso vale desde que a ordenação por y seja mantida (por intercalação) sem nova ordenação de custo superlinear em cada nível; a ordenação inicial custa $\Theta(n \log n)$ uma única vez e não altera o total.

5.2 Por que comparar até sete vizinhos?

Seja δ a menor distância encontrada nas duas metades. Na faixa, só interessam pares cuja diferença vertical é menor que δ. Tomando um ponto p, os candidatos acima dele cabem em um retângulo de largura 2δ (δ para cada lado da divisão) e altura δ. Dividindo esse retângulo em 8 quadrados de lado $\delta/2$, cada quadrado fica inteiramente em uma das metades, onde os pontos distam pelo menos δ entre si. Como a diagonal de um quadrado é $\delta\sqrt{2}/2 \approx 0{,}71\delta < \delta$, cada quadrado contém no máximo um ponto. Logo o retângulo tem no máximo 8 pontos, contando p, e basta comparar cada ponto com os 7 seguintes na ordem crescente de y. O trabalho da combinação é $\Theta(n)$ por nível.

5.3 Testes de corretude

A implementação foi comparada à força bruta em sete situações específicas, além de 1.000 testes aleatórios, totalizando 1.007 testes, todos aprovados. Cada situação de borda exercita um ponto frágil do algoritmo:

- **Dois pontos** e **quatro pontos:** o menor caso possível e o menor caso que já exige recursão e combinação.
- **Pontos duplicados:** a distância mínima é zero, e a faixa de largura 2δ degenera.
- **Pontos alinhados** e **mesmo y:** geram empates na ordenação por x ou por y e testam a divisão pela mediana e a varredura da faixa.
- **Par cruzando a divisão:** a resposta só é encontrada na etapa de combinação.
- **Empate entre pares:** vários pares com a mesma distância mínima; o valor retornado deve coincidir com o da força bruta.

Os testes aleatórios cobrem combinações que os casos manuais não antecipam.

5.4 Resultados experimentais

Tabela 3. Tempos médios da Etapa 2 (ms).

n	Força bruta	Div. e conquista	Razão FB ÷ D&C
100	0,0681	0,2736	0,25
200	0,2705	0,4603	0,59
400	1,0744	0,9572	1,12
800	4,5232	0,8501	5,32
1.600	17,9516	2,3103	7,77
3.200	71,9263	4,3932	16,4
6.400	282,9314	5,6020	50,5

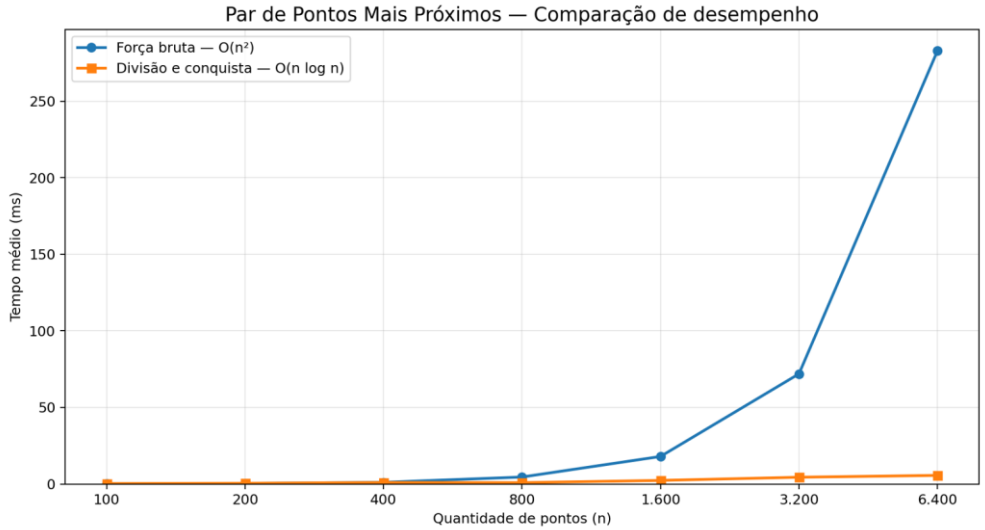


Figura 3. Tempos de força bruta e divisão e conquista.

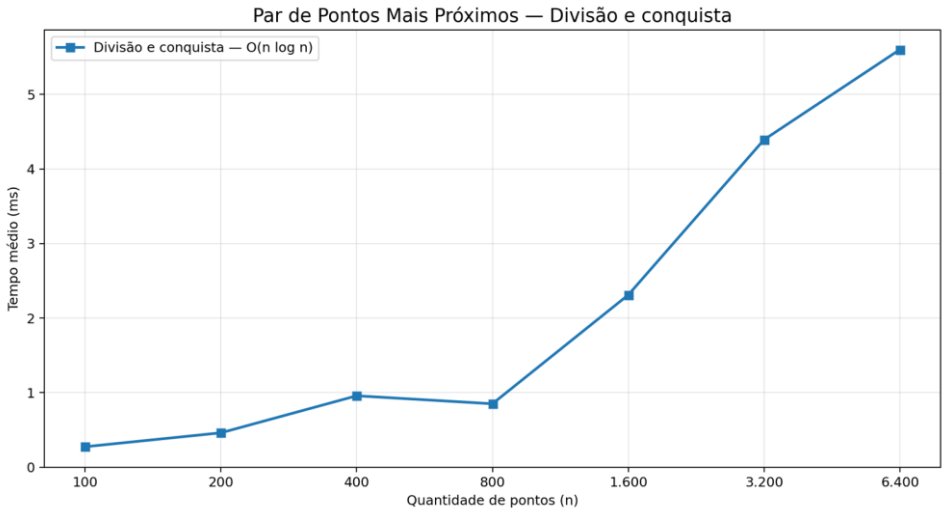


Figura 4. Evolução do desempenho do método de divisão e conquista.

Para n = 100 e 200, a força bruta foi mais rápida. Em n = 400, a divisão e conquista passou a ter vantagem (0,9572 ms contra 1,0744 ms). Em n = 6.400, os tempos foram 5,6020 ms contra 282,9314 ms: vantagem de aproximadamente 50,5 vezes.

Comparação com a teoria. Para a força bruta, dobrar n deveria quadruplicar o tempo; o observado foi 4,0 (1.600 a 3.200: 17,9516 a 71,9263 ms) e 3,9 (3.200 a 6.400: 71,9263 a 282,9314 ms), próximo de $O(n^2)$. A divisão e conquista cresce bem menos (1,9 e 1,3 ao dobrar n), abaixo do fator ≈ 2,2 previsto para $\Theta(n \log n)$, com oscilações em tamanhos intermediários, por exemplo em n = 800, mais rápido que em n = 400. O custo de ordenação, recursão e manutenção da faixa explica a desvantagem inicial para entradas pequenas, e a variabilidade típica de tempos de poucos milissegundos explica as oscilações.

5.5 Escolha do limiar

Foi avaliada a troca da recursão por força bruta em subproblemas pequenos, usando limiares 3, 8, 16, 32 e 64. Cada configuração foi executada cinco vezes por tamanho de entrada. A Tabela 4 mostra os tempos médios, com o menor valor de cada linha destacado.

Tabela 4. Tempos médios (ms) por limiar de troca para força bruta.

n	Limiar 3	Limiar 8	Limiar 16	Limiar 32	Limiar 64
100	0,1445	**0,0889**	0,1005	0,0916	0,0971
800	1,2499	0,8138	0,5321	**0,4858**	0,4928
3.200	1,9954	1,9234	**1,8118**	1,9822	1,8181
6.400	4,4816	4,3419	**3,7693**	3,8121	4,3292

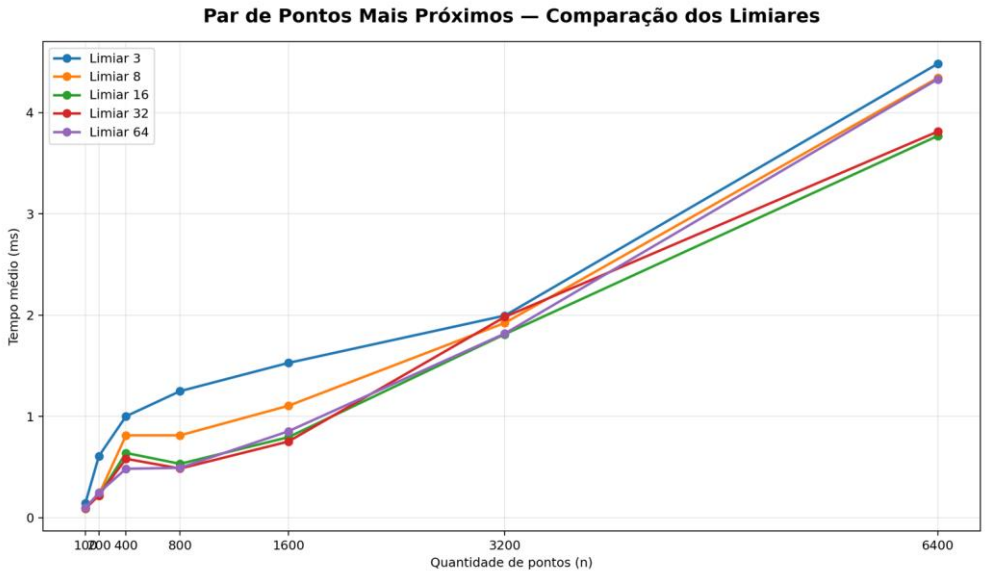


Figura 5. Tempos medidos para os diferentes limiares.

Justificativa. Em subproblemas pequenos, o custo fixo da recursão, da ordenação e da faixa supera o das poucas comparações da força bruta, o que justifica um limiar. Limiar 3 foi o pior (soma de ≈ 7,87 ms nos quatro tamanhos da Tabela 4): quase toda a recursão é mantida. No outro extremo, o limiar 64 piora em n = 6.400 (4,3292 ms contra 3,7693 ms), pois a força bruta passa a custar $\Theta(k^2)$ em cada folha. O limiar 16 teve o menor somatório (≈ 6,21 ms, contra ≈ 6,37 ms do 32 e ≈ 6,74 ms do 64) e também o menor entre as configurações da tabela completa. A diferença entre 16 e 32 é pequena (cerca de 2,5%) e provavelmente

está dentro da variabilidade de cinco repetições; adotamos o 16 como compromisso experimental favorável. O limiar ideal pode variar com hardware, JVM e distribuição das entradas.

6. Síntese dos resultados

As duas etapas ilustram a diferença entre custo assintótico e custo fixo. A força bruta pode vencer para entradas pequenas, mas o crescimento quadrático torna-se desvantajoso quando n aumenta. A divisão e conquista oferece desempenho superior em entradas maiores. Na Etapa 1, o algoritmo de Kadane ainda reduz o custo para Θ(n), destacando que a melhor solução depende da estrutura do problema.

Tabela 5. Comparação entre as duas etapas.

Etapa	Cruzamento	Maior n	FB (ms)	D&C (ms)	Razão
1 — Subarranjo	entre 200 e 400	51.200	285,1313	1,1186	≈ 255×
2 — Pontos	n = 400	6.400	282,9314	5,6020	≈ 50,5×

A Etapa 1 identificou o cruzamento de desempenho entre n = 200 e 400. Na Etapa 2, a divisão e conquista também passou a vencer a partir de n = 400, e o limiar 16 foi escolhido por comparação experimental. Os resultados não constituem prova da complexidade, mas são consistentes com a análise teórica.

7. Arquivos de apoio e execução

Os materiais do trabalho estão organizados no repositório da seguinte forma:

- `data/medicoes_etapa1.csv`, `data/benchmark_pontos.csv` e `data/benchmark_limiares.csv`: dados brutos das medições;
- `figuras/`: imagens e tabelas;
- `src/`: algoritmos em Java;
- `tests/`: testes de corretude.

As medições foram realizadas em Java 21. Os comandos de compilação e execução devem ser mantidos atualizados no README do repositório.

8. Limitações

Os tempos são sensíveis a aquecimento da JVM, resolução do relógio, processos concorrentes e pequenas amostras. Os gráficos e tabelas desta versão consolidada provêm dos experimentos originais de cada etapa, que não foram repetidos na consolidação. Como as duas etapas foram medidas em ambientes diferentes (Seção 2), os tempos absolutos de uma etapa não devem ser comparados diretamente com os da outra.

9. Referências e fontes dos resultados

PEDRO LUCAS DE OLIVEIRA DUARTE. **Etapa 1: Subarranjo de Soma Máxima.** Relatório da equipe, 2026.

EQUIPE. **figuras/tabela_etapa1.md; tabela_etapa2.md; tabela_limiares.md**; gráficos e arquivos CSV de benchmark, 2026.

ENUNCIADO DA DISCIPLINA. **Trabalho Prático: Divisão e Conquista**, 2026.